

Ethan Patterson
Nico Cabrera

Algo HW5 Q2

a.) This is an algorithm that uses BFS to calculate path lengths and then analyze the results in order to generate our conclusion. We are going from S to T, and finding the max paths for them. First we run BFS on both S and T, and store those arrays(named sBFS and tBFS respectively). We can find the shortest distance from S to T by checking the Tth position in sBFS or vice versa. Easy check that it actually exists as well, as long as the value is greater than -1. First, we work with a queue, and load T into it first(working forward and backwards produces the same result, we have to pick one). We have a dynamic programming style "pathcount" array to store the number of paths to T from a given vertex(T gets path count 1 by default), and a "visited" array that tells us if we've already processed a value. Additionally we have set up an adjacency list for our graph to provide ease of access to adjacent vertices. We loop until the queue is empty, grab the current vertex from the queue, and grab every adjacent vertex to it. Then, we loop over each adjacent vertex. First, we check that its sBFS(or tBFS, both work) result was greater than -1, meaning it has an actual connected path to S(and by nature T). Next we check that the tBFS and sBFS result for the adjacent vertex sum to the shortest path. If they don't, that means it's not on the shortest path and can safely be ignored as useless. Path length can't decrease. Lastly, we check that the adjacent vertex's sBFS value and the current vertex's sBFS value - 1 are equal. This ensures that we only feed to nodes on a shallower level, where we can ignore same depth or deeper depth scenarios that may cause problems. If all these conditions are met, the adjacent vertex's pathcount is increased by the current vertex's pathcount. If we haven't "visited" that vertex yet(the vertex has not been the current vertex), we enqueue it and mark it as visited. This will loop over everything BFS style, starting at the deepest relevant node and working upwards, properly summing the paths as we go. Once we finish and the queue is empty, the total path count from S to T is located in our pathcount object at S's position. This total path count marks how many shortest paths exist between S and T.

b.) Pseudocode:

BFS (target_node, vertices, graph):

- Create empty queue

- Enqueue the target node

- Create list of visited nodes

- Visited[target_node - 1] = 0

- While the queue is not empty:

 - Active_vertex = dequeue from queue

 - Create list of adjacent vertices = graph[active_vertex - 1]

 - For neighbors in adjacent vertices:

 - If the neighbor was visited:

 - Visited[neighbor+1] = visited[active_vertex - 1] + 1

 - Enqueue the neighbor

```
Create list of visited vertices
For all vertices:
    If vertex was visited:
        Add vertex to visited vertices
Return visited vertices
```

Number of vertices and edges are input
S and T are input
Graph is a list of edges and input
Adjacency list is constructed from the inputted graph

```
sBFS = BFS(S, vertices, adjacency list)
tBFS = BFS(T, vertices, adjacency list)
```

Shortest path to T = sBFS[T-1]

Check that there exists a shortest path to T

```
Create empty queue
Enqueue T
```

```
Create empty list called path_count (size vertices)
path_count[T-1] = 1
```

```
Create list of visited vertices
Visited[t-1] = 1
```

```
While the queue is not empty
    Active vertex = dequeue
    For all adjacent vertices in adjacency list at active vertex -1:
        If (adjacent vertex can reach S and T AND
            Adjacent vertex part of shortest path to T AND
            Adjacent vertex is shallower than active vertex):
            Path count at the adjacent vertex += path count at the active vertex
            If the adjacent vertex was visited:
                Enqueue the adjacent vertex
                Add the adjacent vertex to the list of visited vertices
```

Output path count at S

c.) The BFS generates our shortest path so we know what to work with. Working from the back, moving up the adjacency list to add to our queue ensures that the “deeper” values are fully completed before the shallower values, ensuring no problems with those. The series of checks: vertex in the BFS tree, sBFS and tBFS summing to shortest path, adjacent vertex is shallower

than current vertex ensure that the path summing only gets done in the right scenario. The classic dynamic programming trick of “start at one end, build from the ground up, use the previous answer to generate the next answer easily” trick works perfectly here.

d.) $O(M + N)$ where M is the number of vertices and N is the number of edges

e.) BFS is by nature $O(M+N)$. The getMaxPaths function simply iterates another M times for its outer while loop(each vertex can only be queued up once), and the inner for loop inside that runs a total of N times(not per iteration, in total). Thus this is also $O(M + N)$. Data input is also $O(N)$ and these all together go to $O(2M + 3N)$ which is still just $O(M+N)$.